UF1 – Ficheros · JAXB · Gson

Persistencia en ficheros – EAC1

Ficheros – File / NIO

Interacción básica con el sistema de archivos (crear, leer, mover, copiar y filtrar).

```
File f = new File(path);       → crear referencia
f.isFile() → ¿es fichero?      f.isDirectory() → ¿es carpeta?
f.isHidden() → ¿oculto?        f.length() → tamaño (bytes)
f.canRead() → ¿lectura?        f.canWrite() → ¿escritura?
f.exists() → ¿existe?
FileFilter ff = f → f.isFile() && f.length() < MAX; → filtro custom
File[] arr = dir.listFiles(ff); → extrae lista filtrada
```

```
long mida = f.length();    // → guarda tamaño ANTES
Files.copy(src, dst, StandardCopyOption.REPLACE_EXISTING); → copia fichero
Files.move(src, dst, StandardCopyOption.REPLACE_EXISTING); → mueve o renombra
```

■ BUG: length() después de move() → devuelve 0.
→ guarda f.length() en var ANTES de mover.

✓ Criterio M → filtro por bytes; Criterio N → prefijo nombre.
Parámetro H opcional → incluye ficheros ocultos.

JAXB – Anotaciones XML

Convierte objetos Java en XML (marshal) y viceversa (unmarshal) mediante anotaciones.

```
@XmlRootElement(name="estacio") → clase raíz
@XmlElement(name="nom")         → elemento hijo XML
@XmlAttribute(name="id")        → atributo XML
@XmlElementWrapper(name="pistes") + @XmlElement(name="pista") → envuelve listas
@XmlTransient                   → ignora campo (no exporta)
```

```
// Marshal (objeto → XML)
JAXBContext ctx = JAXBContext.newInstance(Estacio.class); → inicia contexto
Marshaller m = ctx.createMarshaller(); → motor de escritura
m.setProperty(Marshaller.JAXB_FORMATTED_OUTPUT, true); → tabula XML
m.marshal(estacio, new File("sortida.xml")); → guarda a disco

// Unmarshal (XML → objeto)
Unmarshaller u = ctx.createUnmarshaller(); → motor de lectura
Estacio e = (Estacio) u.unmarshal(new File("entrada.xml")); → lee de disco
```

■ BUG: falta constructor sin args → JAXB falla unmarshal.
→ siempre @XmlRootElement + constructor no-args.

Gson – JSON

Librería de Google para serializar y deserializar objetos Java a formato JSON de forma rápida.

```
@SerializedName("nom_json") → renombra clave individual
new GsonBuilder().setPrettyPrinting().create() → formato legible
```

```
// Serializar
Gson gson = new GsonBuilder().setPrettyPrinting().create(); → inicializa
String json = gson.toJson(estacio); → objeto a String JSON

// Deserializar – objeto
Estacio e = gson.fromJson(json, Estacio.class); → String JSON a objeto

// Deserializar – lista (TypeToken OBLIGATORIO)
Type t = new TypeToken<List<Pista>>(){}.getType(); → extrae tipo genérico
List<Pista> ps = gson.fromJson(json, t); → JSON a lista objetos
```

■ BUG: lista sin TypeToken → ClassCastException.
→ new TypeToken<List<Pista>>(){}.getType() SIEMPRE.

Excepciones – Patrón gestor

Patrón para capturar y encapsular errores de forma centralizada sin perder información.

```
public Estacio obteEstacio(String id) {
 try {
  // operación propensa a fallar
 } catch (Exception e) { // ¡NO solo IOException! → atrapa todo
  throw new GestorExcepcio(e.getMessage()); → relanza controlada
 }
}
```

■ BUG: catch(IOException) solo → NPE escapa sin capturar.
→ catch(Exception e) para capturarlo todo.

Nombres Java ↔ XML/JSON (EAC1)

Reglas de mapeo entre nombres de Java (camelCase) y formatos estructurados (kebab-case/snake_case).

```
altitudMaxima → altitud-maxima      (@XmlElement)
estatObertura → estat-obertura-percentatge
gruixNeu      → gruix_neu           (@SerializedName)
```

✓ Comprueba nombres exactos en el .xml/.json de prueba del prof.
Javadoc: @param @return @throws + @author joan.

UF2 – JDBC · JPA / EclipseLink

Persistencia en BDR – EAC2

JDBC – Conexión y consultas

API nativa de bajo nivel para conectarse y ejecutar consultas SQL en bases de datos relacionales.

```
Connection con = DriverManager.getConnection(
 "jdbc:postgresql://localhost/spring2526s2","ioc","ioc"); → conecta a BD
// SELECT
PreparedStatement ps = con.prepareStatement(
 "SELECT * FROM pista WHERE id = ?"); → prepara query segura
ps.setString(1, id);    // ¡índice empieza en 1! → asigna valor a '?'
ResultSet rs = ps.executeQuery(); → ejecuta lectura (SELECT)
while (rs.next()) { → itera filas
    String nom = rs.getString("nom"); → extrae columna
}
// INSERT / UPDATE / DELETE
ps.executeUpdate();     // ¡no executeQuery! → ejecuta modificación
```

■ BUG: ps.setString(0, id) → índice JDBC empieza en 1, no 0.
BUG: executeQuery() para UPDATE → error de ejecución.
→ executeUpdate() para INSERT / UPDATE / DELETE.

✓ Cierra: rs → ps → con. O try-with-resources.

JPA – Herencia SINGLE_TABLE

Estrategia ORM para mapear toda una jerarquía de clases polimórficas (herencia) en una única tabla.

```
@Entity → es tabla BD
@Inheritance(strategy = InheritanceType.SINGLE_TABLE) → todo en 1 tabla
@DiscriminatorColumn(name = "tipus") → columna diferenciadora
@Table(name = "pista") → nombre físico tabla
public abstract class Pista {
 @Id @Column(length=14) private String id; → PK
 @Column(length=100)    private String nom;
 @Column(length=50)     private String comarca;
 // constructor no-args OBLIGATORIO para JPA
}
@Entity @DiscriminatorValue("ALPINA") → valor if alpina
public class PistaAlpina extends Pista { … }
@Entity @DiscriminatorValue("NORDICA") → valor if nordica
public class PistaNordica extends Pista { … }
@Entity @DiscriminatorValue("RAQUETES") → valor if raquetes
public class ItinerariRaquetes extends Pista { … }
```

■ BUG: @AllArgsConstructor en subclase sin @NoArgsConstructor …
→ JPA no puede instanciar en consultas.
→ añade @NoArgsConstructor a todas las subclases.

JPA – Relación Estacio ↔ Pista

Mapeo de relaciones bidireccionales (1:N y N:1) entre entidades en la base de datos.

```
// Estacio.java
@OneToMany(mappedBy="estacio", cascade=CascadeType.ALL) → lado 1
private List<Pista> pistes = new ArrayList<>();

// Pista.java
@ManyToOne → lado N (propietario)
@JoinColumn(name="estacio_id") → clave foránea física
private Estacio estacio;
```

■ BUG: hashCode() de Pista usa this.estacio completo
→ StackOverflowError por referencia circular.
→ usa this.estacio.getId() dentro de hashCode().

JPA – JPQL y operaciones

Lenguaje de consultas orientado a objetos de JPA, independiente del dialecto SQL subyacente.

```
// atributos camelCase Java, ¡NO nombres columna SQL!
em.createQuery(
 "SELECT p FROM Pista p WHERE p.oberta = true") → crea JPQL
 .getResultList(); → devuelve lista objetos

// UPDATE
em.createQuery(
 "UPDATE Pista p SET p.gruixNeu = p.gruixNeu + :inc
  WHERE p.id = :id")
 .setParameter("inc", increment) → enlaza variable
 .setParameter("id", id)
 .executeUpdate(); → aplica cambios

EntityTransaction tx = em.getTransaction(); → transacciones
tx.begin(); /* op */ tx.commit(); → inicio/fin
```

■ BUG: JPQL "gruix_neu" (SQL) → debe ser "gruixNeu" (Java).
BUG: actualitzaGruixNeu recibe INCREMENT, no valor absoluto.

✓ persist() → entidad nueva; merge() → ya existente.
find(Pista.class, id) → null si no existe.
persistence.xml: unidad "eac2PU", provider EclipseLink.

UF3 – BaseX · XQuery · XQUF

BD documental nativa – EAC3 Ex1

Conexión BaseX (XQJ)

API estándar (XQJ) para establecer conexión con una base de datos documental XML nativa como BaseX.

```
SednaConnection con = new SednaConnection(
 "localhost", 1984, "pistes", "admin", "admin");
con.connect(); → conecta al puerto BaseX
XQDataSource ds = con.getXQDataSource(); → origen datos
XQConnection xqc = ds.getConnection(); → sesión XQJ
XQExpression expr = xqc.createExpression(); → motor queries
```

✓ Puerto BaseX: 1984. BD inicial: <pistes></pistes> vacía.
Tests pueblan la BD dinámicamente – no asumas datos.

XQuery – Consultas (executeQuery)

Lenguaje funcional para realizar consultas y extraer datos de nodos jerárquicos en documentos XML.

```
// Por id
String q = "doc('pistes')/pistes/pista[@id='" + id + "']";
XQResultSequence rs = expr.executeQuery(q); → lanza query lectura
if (rs.next()) { → ¿hay resultado?
 Pista p = Utilitats.xmlToPista(rs.getItemAsString(null)); → parsea nodo
}

// Por filtro de remonte
String q2 = "doc('pistes')/pistes/pista"
 + "[remuntadors/remuntador='" + tipus + "']";
XQResultSequence rs2 = expr.executeQuery(q2); → lanza query
List<Pista> list = new ArrayList<>();
while (rs2.next()) → recorre cursor XML
 list.add(Utilitats.xmlToPista(rs2.getItemAsString(null))); → guarda a lista
```

XQUF – Modificaciones (executeUpdate)

Extensión de XQuery (XML Query Update Facility) para insertar, actualizar y borrar nodos de la base de datos.

```
// INSERT node
String xml = Utilitats.pistaToXml(pista); → serializa a string
String ins = "insert node " + xml
 + " into doc('pistes')/pistes";
expr.executeUpdate(ins); // ¡NO executeQuery! → lanza inserción

// DELETE node
String del = "delete node doc('pistes')/pistes/pista[@id='" + id + "']";
expr.executeUpdate(del); → lanza borrado
```

■ BUG: executeQuery() para XQUF → error de ejecución.
→ INSERT / DELETE / REPLACE → executeUpdate() SIEMPRE.

Teoría BD documentales – Preguntas teóricas PAF

Comparativa teórica entre las capacidades de las bases de datos XML nativas vs las relacionales con columnas XML.

BD XML nativa:
Almacena docs XML como unidad atómica.
Consultas: XQuery/XPath (no SQL). Esquema: opcional.
Ventajas: jerarquía profunda, estructura variable.
Inconvenientes: menos estándar, peor en tablas.

BD relacional + columna XML:
Estructura primaria relacional; XML dentro de columnas.
SQL + XMLEXISTS / XMLQUERY. Mejor si mayoría tabulares.

XPath: navegación árbol XML (expresiones de ruta).
XQuery: incluye XPath + FLWOR (for/let/where/order/return).
XQUF: extensión para insert / delete / replace node.

XPath – Expresiones básicas

Lenguaje de rutas simple para navegar y ubicar rápidamente elementos y atributos en el árbol XML.

```
/pistes/pista → extrae todos los hijos <pista>
/pistes/pista[@id="p1"] → filtra por atributo exacto
/pistes/pista[nom="Bosc"] → filtra por valor de elemento hijo
/pistes/pista[remuntadors/remuntador="Telecadira"] → filtro jerárquico
count(/pistes/pista) → cuenta la cantidad de nodos
/pistes/pista/nom/text() → extrae el texto (string) del nodo
```

FLWOR

Estructura avanzada en XQuery (For, Let, Where, Order by, Return) para iterar, filtrar y transformar resultados.

```
for $p in doc("pistes")/pistes/pista → iteración
where $p/oberta = "true" → condición / filtro
order by $p/nom → ordenación
return $p → resultado a escupir
```

Clase Utilidades (ya implementada)

Métodos de utilidad pre-proporcionados en el examen para transformar objetos Pistas en XML y viceversa.

```
Utilitats.pistaToXml(Pista p) → serializa: objeto a String XML
Utilitats.xmlToPista(String xml) → parsea: String XML a objeto
```

✓ No reimplementes Utilidades. Úsala para Pista ↔ XML.

NO ENTRA en la PAF (excluidos oficiales)

Lista explícita de tecnologías, librerías y APIs que NO serán evaluadas en el examen práctico final.

✗ DB4O ✗ XML:DB API ✗ XUpdate
✗ ObjectDB ✗ JavaBeans + PropertyChangeListener

UF4 – Spring Data JPA · Lombok · Errores clave PAF

Componentes de acceso a datos – EAC3 Ex2

Spring Data JPA – Repositorio

Capa de abstracción sobre JPA que autogenera consultas e implementaciones al extender interfaces de Spring.

```
public interface PistaRepository
 extends JpaRepository<Pista, String> { → extiende CRUD base

 // Query method automatic
 List<Pista> findByOberta(boolean oberta); → genera SQL por nombre

 // JPQL personalizado
 @Query("SELECT p FROM Pista p WHERE p.gruixNeu ≥ :g") → JPQL custom
 List<Pista> findByGruixMinim(@Param("g") int gruix); → asocia parámetro

 // Modificación – DOBLE anotación OBLIGATORIA
 @Modifying → habilita alteración (UPDATE/DELETE)
 @Transactional → empaqueta en transacción
 @Query("UPDATE Pista p SET p.gruixNeu = p.gruixNeu + :inc WHERE p.id = :id")
 int actualitzaGruix(@Param("id") String id,
                     @Param("inc") int inc); → ejecuta y devuelve nº filas
}
```

■ BUG: @Modifying sin @Transactional → TransactionRequired …
→ las DOS anotaciones juntas para UPDATE/DELETE.

Métodos JpaRepository

Operaciones CRUD básicas (guardar, buscar, borrar) disponibles automáticamente sin escribir SQL ni JPQL.

```
save(entity)   → (guarda/actualiza) INSERT o UPDATE
findById(id)   → (busca por PK) devuelve Optional<T>
findAll()      → (busca todo) devuelve List<T>
deleteById(id) → (borra) ejecuta DELETE
existsById(id) → (comprueba) devuelve boolean
count()        → (cuenta filas) devuelve long
```

✓ findById → Optional: .orElse(null) o .isPresent().
save() hace merge si entidad ya tiene id asignado.

Arquitectura EAC3 Ex2 (en capas)

Estructura limpia del proyecto separando responsabilidades: Controlador, Lógica de Negocio, DAO y Datos.

```
Controller / TestGestors
 ■■ GestorPista (lógica de negocio) → reglas y validaciones
   ■■ PistaDAO (implementación DAO) → abstracción BD
     ■■ PistaRepository (Spring Data) → acceso físico
       ■■ PostgreSQL → base de datos final
```

✓ Gestor NO accede directamente al repo; usa el DAO.
Componente = interfaz pública + impl. encapsulada.

Lombok – Anotaciones seguras con JPA

Librería para autogenerar código (getters, setters), con precauciones esenciales al usarla en entidades JPA.

```
@Entity @Table(name="pista")
@Getter @Setter → autogenera getters/setters base
@NoArgsConstructor // JPA OBLIGATORIO → constructor vacío
@AllArgsConstructor → constructor completo
@ToString(exclude="estacio") // evita ciclo infinito → excluye prop relacional
public class Pista { … }

// @Data = @Getter+@Setter+@EqualsAndHashCode+@ToString
// → PELIGRO con relaciones bidireccionales (StackOverflow)
// Subclases: NO @AllArgsConstructor sin @NoArgsConstructor
```

■ BUG: @Data en clase JPA con rel. bidireccional →
StackOverflowError por hashCode/toString recursivo.
→ usa @Getter @Setter @NoArgsConstructor por separado.

Errores clave PAF – Los 10 bugs reales

Resumen vital de los 10 errores y confusiones más comunes que causan pérdidas de tiempo o fallos en el examen final.

■ Files.move() y length(): guarda length() ANTES del move.

■ Constructor no-args: obligatorio para JAXB y JPA.

■ hashCode() circular: usa getId(), no el objeto completo.

■ @Modifying: siempre acompañado de @Transactional.

■ TypeToken Gson: listas SIEMPRE con TypeToken.

■ JPQL camelCase: gruixNeu, NO gruix_neu (SQL).

■ XQUF executeUpdate(): NUNCA executeQuery para INSERT/DELETE.

■ JDBC index 1: setString(1, val), NO setString(0, val).

■ catch(Exception e): no catch(IOException) solo.

■ actualitzaGruixNeu: recibe INCREMENT, no valor absoluto.

✓ PAF ~90 min | ~22 min/UF. No te atasques. Código parcial suma…
Tienes lista oficial de métodos (sin javadoc) en el examen.